

LOVE IN THE RUIN — Full App Development Sheet

LOVE IN THE RUIN - Full App Development Sheet

A complete, end-to-end description of what this app **is**, **does**, and **how it works** - backend, frontend, data structures, behaviors, and flow.

1. Purpose of the App

This application is a **single-song, multi-stem, section-aware lyric visualizer**. It synchronizes:

- Audio stems (6 isolated WAV files)
- Lyric timing
- Beat grid
- Production cues
- Visual behaviors

The result is a cinematic, reactive, music-driven experience where every section of the song has its own visual identity.

2. Backend Architecture

The backend serves **audio**, **metadata**, and **timing information** to the frontend.

2.1 Stems (6 WAV Files)

Stored in /backend/stems/:

- vocals.wav
- drums.wav
- bass.wav
- melody.wav
- harmonies.wav
- fx.wav

Each stem is streamed individually so the frontend can:

- Visualize each waveform separately
- React to production cues
- Animate based on specific stems

2.2 Metadata Files

Stored in /backend/metadata/.

lyrics.json

Contains:

- Song title
- BPM
- Sections (intro, hook, verse, bridge, outro)
- Speaker (male, female, sample)
- Production cues (beat drop, ambience, filtered, etc.)
- Lines and word-level timestamps

This file drives:

- Section transitions
- Word highlighting
- Typography changes
- Visual effects

beats.json

Contains:

- BPM
- Array of beat timestamps

Used for:

- Pulsing waveform
- Word accent glow
- Helicopter light oscillation
- Timing-accurate animations

waveform.json

Contains downsampled amplitude arrays for each stem:

- drums
- bass
- vocals
- melody
- harmonies
- fx

Used for:

- Drawing waveforms
 - Stem-specific animations
 - FX wobble
-

3. Frontend Architecture

The frontend is a **reactive visualizer** that responds to:

- Current section
- Current line
- Current word
- Beat grid
- Production cues
- Speaker identity

3.1 Section-Aware Visualizer

Each section type changes the visual style.

Hooks:

- Bright glow colors
- Tight waveform
- Rhythmic spacing

Verses:

- Documentary feel
- Gritty textures
- Lower contrast

Bridge:

- Airy spacing
- Softer glow
- Floating motion

Intro/Outro:

- Cinematic
- Filtered textures
- Slow movement

3.2 Production Cue Reactions

Examples:

helicopter_distant:

- Background oscillation
- Low-frequency wobble in FX waveform

beatdrop:

- Waveform tightens
- Colors intensify

filtered:

- Typography softens
- Waveform blurs slightly

3.3 Speaker-Based Styling

Male:

- Warmer colors
- Heavier typography

Female:

- Cooler colors
- Lighter typography

Sample:

- Grainy texture
- Vintage color palette

3.4 Word-Level Behavior

When a word has "accent": true:

- Cyan glow
- Scale up
- Pulse on beat

4. Frontend Flow

1. Fetch /lyrics
2. Fetch /beats
3. Fetch /waveform
4. Load main audio file
5. As audio plays:

- Determine current section
 - Render section label
 - Render current line
 - Highlight active word
 - Animate waveform
 - Trigger production cues
-

5. What the App *Is*

- A **music-driven visual storytelling engine**
 - A **section-aware lyric visualizer**
 - A **multi-stem audio playback system**
 - A **timing-accurate animation platform**
 - A **cinematic UI experience**
-

6. What the App *Does*

- Loads and plays 6 isolated stems
 - Displays synchronized lyrics
 - Highlights words in real time
 - Animates waveforms per stem
 - Reacts visually to production cues
 - Changes typography based on speaker
 - Changes color palette per section
 - Pulses visuals on beat
 - Creates a fully immersive music experience
-

7. What You Choose Next

To continue development, choose the first section to prototype:

- Intro sample
- First Hook
- Verse 1

- Bridge
- Final Hook

Once selected, the system will generate:

- Exact JSON
- Frontend code
- Visual behavior
- Timing model

This document fully describes the app's architecture, purpose, and behavior.